

Directory Entry Caching for Filesystem (R0)

Fixing issue [2663](#), *Enable efficient retrieval of file size from directory_entry* and issue [2677](#), *directory_entry::status is not allowed to be cached ...*

This paper provides two proposals for solving the problem of efficiently caching state information obtained during directory iteration. Guidance is sought as to which proposal is preferred.

Both proposals include full wording, have been implemented, and avoid use of mutable data members. Both offer guidance to users on how to use or not use cached information as desired, without exposing to the user whether information is actually cached. Both allow the user to write code that is fully portable between implementations that cache or do not cache.

Proposal 1 exposes what information may possibly be cached, and so would require additions to the standard library should future operating systems or file systems appear that supply additional state on directory iteration. It is conceptually simpler and closer to existing practice than proposal 2.

Proposal 2 does not expose what state may possibly be cached, and so requires no changes to the standard library as operating systems and file systems evolve. But this proposal is harder to reason about and depends on automatic conversions that may cause concern.

Background

Directory iteration in real-world operating systems always returns directory state information containing at least the file name. POSIX has an option to also return file status, but not all popular distributions implement this. Windows always returns file status, file size, and last modification date. Accessing this additional state from the directory entry is much more efficient than re-accessing the file system to obtain it. Users know this and expect the standard library filesystem to deliver the same efficiency.

The initial filesystem TS proposal and the Boost Filesystem implementation limited the additional state stored by class `directory_entry` to the regular status and symlink status, since these were the only additional elements common to several operating systems. The caching of this additional information was described using mutable exposition-only data members. The LWG removed the mutable members and associated caching wording because mutable members are problematic and race prone in multi-threaded environments. The original design also exposed too many implementation details and was not easily extendible to additional cache information such as file size and last write timestamp.

When `directory_entry` caching was removed from the TS, the LWG promised to revisit the issue when Filesystem was added to the standard. Two issues were subsequently filed that remind us of that promise.

LWG issue [2663](#), *Enable efficient retrieval of file size from directory_entry*, requests that for Windows caching be extended to file size.

LWG issue [2677](#), *directory_entry::status is not allowed to be cached as a quality-of-implementation issue*, requests the reinstatement of permission for implementations to cache directory entry information.

Design decisions

Add cache refresh() functions

The original TS design conflated the observer functions that access cached state information with refreshing the cached state. Since refreshing the cached state is non-const, the cached member data had to be mutable and that was unacceptable. The new design for both proposals provides

separate non-const refresh functions that are called by all other non-const functions that modify the stored path, ensuring cache integrity. The refresh functions can be called by users if desired to refresh stale cached data.

Proposal 1 - Expose the attributes that may be cached

With the separation of refresh and observer functionality, the traditional observer functions become truly const and provide the model for expanding caching to cover file size and last write time. But this approach requires the standard library be changed if future operating systems add new attributes that become caching candidates.

Proposal 2 - Hide the attributes that may be cached

By allowing implementations to add overloads to operational functions taking a const `directory_entry&` argument instead of a const `path&` argument, any file attribute the operating system exposes can be cached. Because there is an automatic conversion from const `directory_entry&` to const `path&`, user code works whether or not the attribute is cached. That eliminates the need for the standard library to expose which attributes may be cached. It just works. But this approach may be more confusing and lacks existing practice.

Proposal 1 proposed wording

27.10.12 Class `directory_entry` [class.directory_entry]

```
namespace std::filesystem {
class directory_entry {
public:
    // constructors and destructor
    directory_entry() noexcept = default;
    directory_entry(const directory_entry&) = default;
    directory_entry(directory_entry&&) noexcept = default;
    explicit directory_entry(const path& p);
    directory\_entry\(const path& p, error\_code& ec\);
    ~directory_entry();

    // modifiers
    directory_entry& operator=(const directory_entry&) = default;
    directory_entry& operator=(directory_entry&&) noexcept = default;
    void assign(const path& p);
    void assign\(const path& p, error\_code& ec\);
    void refresh();
    void refresh\(error\_code& ec\) noexcept;
    void replace_filename(const path& p);

    // observers
    const path& path() const noexcept;
    operator const path&() const noexcept;
    file_status status() const;
    file_status status(error_code& ec) const noexcept;
    file_status symlink_status() const;
    file_status symlink_status(error_code& ec) const noexcept;
    uintmax_t file_size() const;
    uintmax\_t file\_size\(error\_code& ec\) const noexcept;
    file_time_type last_write_time() const;
    file\_time\_type last\_write\_time\(error\_code& ec\) const noexcept;

    bool operator< (const directory_entry& rhs) const noexcept;
    bool operator==(const directory_entry& rhs) const noexcept;
    bool operator!=(const directory_entry& rhs) const noexcept;
    bool operator<=(const directory_entry& rhs) const noexcept;
    bool operator> (const directory_entry& rhs) const noexcept;
    bool operator>=(const directory_entry& rhs) const noexcept;
private:
    path m_path; // for exposition only
};
}
```

A `directory_entry` object stores a path object.

Implementations are permitted and encouraged to store values for status, symlink status, file size, and last write time attributes in `directory_entry` objects if such values are available during directory iteration and storing them would allow the implementation to eliminate file system accesses by `directory_entry` observer functions (`ifs.op.funcs`). Such stored attribute values are said to be *cached*.

[Example:

```
using namespace std::filesystem;

for (auto&& itr : directory_iterator(p))
{
    // use possibly cached last write time to minimize disk accesses
    std::cout << itr.path() << " " << itr.last_write_time() << "\n";
}

for (auto&& itr : directory_iterator(p))
{
    ... potentially lengthy computations ...
    // do not use cached last write time since the cache may be stale
    std::cout << itr.path() << " " << last_write_time(*itr) << "\n";
}
```

On implementations that do not cache the last write time, both loops will result in a potentially expensive call to the `std::filesystem::last_write_time` function.

On implementations that do cache the last write time, the first loop will use the cached value and so will not result in a potentially expensive call to the `std::filesystem::last_write_time` function.

The code is portable to any implementation, regardless of whether or not it employs caching.

—end example]

27.10.12.1 directory_entry constructors [directory_entry.cons]

```
explicit directory_entry(const path& p);
directory_entry(const path& p, error_code& ec);
```

Effects: Constructs an object of type `directory_entry`, then `refresh()` or `refresh(ec)`, respectively.

Postcondition: `path() == p`.

Throws: As specified in Error reporting ([fs.err.report]).

27.10.12.2 directory_entry modifiers [directory_entry.mods]

```
void assign(const path& p);
void assign(const path& p, error_code& ec);
```

Effects: Equivalent to `m_path = p`, then `refresh()` or `refresh(ec)`, respectively.

Postcondition: `path() == p`.

Throws: As specified in Error reporting ([fs.err.report]).

```
void refresh();
void refresh(error_code& ec) noexcept;
```

Effects: Stores the current value for any cached attribute values of the file `p` resolves to.

Throws: As specified in Error reporting ([fs.err.report]).

```
void replace_filename(const path& p);
```

Effects: Equivalent to `m_path = parent_path() / p`, then `refresh()`.

Postcondition: `path() == x.parent_path() / p` where `x` is the value of `path()` before the function is called.

27.10.12.3 directory_entry observers [directory_entry.obs]

```
const path& path() const noexcept;
operator const path&() const noexcept;
```

Returns: `m_path`

```
file_status status() const;
file_status status(error_code& ec) const noexcept;
```

Returns: If cached, the status attribute value. Otherwise, `status(path())` or `status(path(), ec)`, respectively.

Throws: As specified in Error reporting ([fs.err.report]).

```
file_status symlink_status() const;  
file_status symlink_status(error_code& ec) const noexcept;
```

Returns: If cached, the symlink status attribute value. Otherwise, symlink_status(path()) or symlink_status(path(), ec), respectively.

Throws: As specified in Error reporting ([fs.err.report]).

```
uintmax_t file_size() const;  
uintmax_t file_size(error_code& ec) const noexcept;
```

Returns: If cached, the file size attribute value. Otherwise, file_size(path()) or file_size(path(), ec), respectively.

```
file_time_type last_write_time() const;  
file_time_type last_write_time(error_code& ec) const noexcept;
```

Returns: If cached, the last write time attribute value. Otherwise, last_write_time(path()) or last_write_time(path(), ec), respectively.

```
bool operator==(const directory_entry& rhs) const noexcept;
```

Returns: m_path == rhs.m_path.

```
bool operator!=(const directory_entry& rhs) const noexcept;
```

Returns: m_path != rhs.m_path.

```
bool operator< (const directory_entry& rhs) const noexcept;
```

Returns: m_path < rhs.m_path.

```
bool operator<=(const directory_entry& rhs) const noexcept;
```

Returns: m_path <= rhs.m_path.

```
bool operator> (const directory_entry& rhs) const noexcept;
```

Returns: m_path > rhs.m_path.

```
bool operator>=(const directory_entry& rhs) const noexcept;
```

Returns: m_path >= rhs.m_path.

Proposal 2 proposed wording

27.10.12 Class directory_entry [class.directory_entry]

```
namespace std::filesystem {  
    class directory_entry {  
    public:  
        // constructors and destructor  
        directory_entry() noexcept = default;  
        directory_entry(const directory_entry&) = default;  
        directory_entry(directory_entry&&) noexcept = default;  
        explicit directory_entry(const path& p);  
        directory_entry(const path& p, error_code& ec);  
        ~directory_entry();  
  
        // modifiers  
        directory_entry& operator=(const directory_entry&) = default;  
        directory_entry& operator=(directory_entry&&) noexcept = default;  
        void assign(const path& p);  
        void assign(const path& p, error_code& ec);  
        void refresh();  
        void refresh(error_code& ec) noexcept;  
        void replace_filename(const path& p);  
  
        // observers  
        const path& path() const noexcept;  
        operator const path&() const noexcept;  
        file_status status() const;  
        file_status status(error_code& ec) const noexcept;  
        file_status symlink_status() const;  
        file_status symlink_status(error_code& ec) const noexcept;  
  
        bool operator< (const directory_entry& rhs) const noexcept;  
        bool operator==(const directory_entry& rhs) const noexcept;  
        bool operator!=(const directory_entry& rhs) const noexcept;  
        bool operator<=(const directory_entry& rhs) const noexcept;
```

```

bool operator> (const directory_entry& rhs) const noexcept;
bool operator>=(const directory_entry& rhs) const noexcept;
private:
    path m_path; // for exposition only
};
}

```

A `directory_entry` object stores a path object.

Implementations are permitted and encouraged to store values for file system attributes in `directory_entry` objects if such values are available during directory iteration and storing them would improve efficiency of operational query functions ([fs.op.funcs]). Such stored attribute values are said to be cached. Implementations are permitted and encouraged to overload operational query functions that take a `const path&` argument with an additional signature replacing the argument with a `const directory_entry&` argument if the implementation of that added signature can be made more efficient by using cached attribute values. Such an overload shall be equivalent to ([structure.specifications]) the `const path&` overload of the function except that attribute values shall be obtained from the `directory_entry`'s cached values rather than the external file system.

[Example:

```

using namespace std::filesystem;

for (auto&& itr : directory_iterator(p))
{
    // use possibly cached last write time to minimize disk accesses
    std::cout << itr.path() << " " << last_write_time(*itr) << '\n';
}

for (auto&& itr : directory_iterator(p))
{
    ... potentially lengthy computations ...
    // do not use cached last write time since the cache may be stale
    std::cout << itr.path() << " " << last_write_time(itr->path()) << '\n';
}

```

On implementations that do not cache the last write time, both loops will result in a potentially expensive call to the `std::filesystem::last_write_time` function.

On implementations that do cache the last write time, the first loop will use the cached value and so will not result in a potentially expensive call to the `std::filesystem::last_write_time` function.

The code is portable to any implementation, regardless of whether or not it employs caching.

—end example]

27.10.12.1 `directory_entry` constructors [directory_entry.cons]

```

explicit directory_entry(const path& p);
directory_entry(const path& p, error_code& ec);

```

Effects: Constructs an object of type `directory_entry`, then `refresh()` or `refresh(ec)`, respectively.

Postcondition: `path() == p`.

Throws: As specified in Error reporting ([fs.err.report]).

27.10.12.2 `directory_entry` modifiers [directory_entry.mods]

```

void assign(const path& p);
void assign(const path& p, error_code& ec);

```

Effects: Equivalent to `m_path = p`, then `refresh()` or `refresh(ec)`, respectively.

Postcondition: `path() == p`.

Throws: As specified in Error reporting ([fs.err.report]).

```

void refresh();
void refresh(error_code& ec) noexcept;

```

Effects: Stores the current value for any cached attribute values of the file `p` resolves to.

Throws: As specified in Error reporting ([fs.err.report]).

```

void replace_filename(const path& p);

```

Effects: Equivalent to `m_path = parent_path()/ p`, then `refresh()`.

Postcondition: `path() == x.parent_path() / p` where `x` is the value of `path()` before the function is called.

27.10.12.3 directory_entry observers [directory_entry.obs]

```
const path& path() const noexcept;  
operator const path&() const noexcept;
```

Returns: `m_path`

```
file_status status() const;  
file_status status(error_code& ec) const noexcept;
```

Returns: `status(path()[l, ec])`.

Throws: As specified in Error reporting (`[fs.err.report]`).

```
file_status symlink_status() const;  
file_status symlink_status(error_code& ec) const noexcept;
```

Returns: `symlink_status(path()[l, ec])`.

Throws: As specified in Error reporting (`[fs.err.report]`).

```
bool operator==(const directory_entry& rhs) const noexcept;
```

Returns: `m_path == rhs.m_path`.

```
bool operator!=(const directory_entry& rhs) const noexcept;
```

Returns: `m_path != rhs.m_path`.

```
bool operator<(const directory_entry& rhs) const noexcept;
```

Returns: `m_path < rhs.m_path`.

```
bool operator<=(const directory_entry& rhs) const noexcept;
```

Returns: `m_path <= rhs.m_path`.

```
bool operator>(const directory_entry& rhs) const noexcept;
```

Returns: `m_path > rhs.m_path`.

```
bool operator>=(const directory_entry& rhs) const noexcept;
```

Returns: `m_path >= rhs.m_path`.

Operating system APIs

For Windows, `directory_entry` information is obtained by calling function [FindFirstFile](#), [FindFirstFileEx](#), or [FindNextFile](#), which return pointers to structure [WIN32_FIND_DATA](#), and can be refreshed by calling function [GetFileInformationByHandle](#) which returns a pointer to structure [BY_HANDLE_FILE_INFORMATION](#).

For some POSIX-like systems, such as [Linux](#) and some BSD-based distributions, glibc versions since 2.19 may support an additional struct dirent field named `d_type` "making it possible to avoid the expense of calling `lstat`". POSIX specifies that the macro `_DIRENT_HAVE_D_TYPE` is defined if `d_type` is present.

References

Issue 2663, *Enable efficient retrieval of file size from directory_entry*,
cplusplus.github.io/LWG/lwg-active.html#2663

Issue 2677, *directory_entry::status is not allowed to be cached as a quality-of-implementation issue*,
cplusplus.github.io/LWG/lwg-active.html#2677

N4582, *Working Draft, Standard for Programming Language C++, 2016*,
www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/n4582.pdf

N4100, *Programming Languages — C++ — File System Technical Specification, 2014*,
www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n4100.pdf

Boost Filesystem Library, V3, 2015,
www.boost.org/doc/libs/1_60_0/libs/filesystem/doc/index.htm
